

CUDA & GPGPU Programming Architecture Cheatsheet

L0 Essence: The essence of GPU parallel computing is to achieve high-throughput data parallel processing and tensor computation acceleration through massive lightweight threads mapping under a 3D grid-block-thread topology, executing on SIMT (Single Instruction Multiple Threads) hardware architecture coupled with high-bandwidth memory (HBM/GDDR).

[M1.1] Grid-Block-Thread Hierarchy Model

Core Definition: The CUDA software programming model organizes threads hierarchically into a Grid of thread Blocks. Each Block contains up to 1024 threads, sharing on-chip memory and supporting barrier synchronization. Bottom Mechanism: Thread indices map to physical data coordinates. The 1D mapping index formula is: `int tid = blockIdx.x * blockDim.x + threadIdx.x;` For 2D spatial layouts, coordinates are mapped using: `int x = blockIdx.x * blockDim.x + threadIdx.x;` `int y = blockIdx.y * blockDim.y + threadIdx.y;`. This model governs GPU execution mapping.

[M1.2] Kernel Device Life Cycle

Core Definition: A Kernel is a parallel function declared to execute on the GPU device, launched asynchronously from the host CPU. Bottom Mechanism: When a CPU thread calls a Kernel with configuration parameters `<<grid, block, sharedMem, stream>>`, the launch request is pushed into a hardware command queue (CUDA Stream). The CPU returns immediately, while the GPU hardware scheduler performs Grid Launch to dispatch Blocks.

[M1.3] Streaming Multiprocessor (SM) Physical Architecture

Core Definition: The Streaming Multiprocessor (SM) is the fundamental hardware block of the GPU's processing power. Bottom Mechanism: Each SM houses multiple SPs (Single Precision CUDA Cores), DP Cores (Double Precision), SFUs (Special Function Units for transcendental math like sin/cos), and a massive Register File (RF). The warp scheduler inside the SM dispatches instruction packets to execution units.

[M1.4] SIMT Architecture & Warp Scheduling Logic

Core Definition: Single Instruction Multiple Threads (SIMT) is the hardware execution model where the GPU manages and executes threads in groups of 32 called Warps. Bottom Mechanism: A Warp is the smallest execution and scheduling unit in the SM. The 32 threads in a Warp share a single Program Counter (PC) and execute the same instruction in lockstep. The Warp Scheduler switches active Warps instantly to hide execution stalls.

[M1.5] Warp Divergence Mitigation

Core Definition: Warp Divergence occurs when threads in a single Warp branch to different execution paths due to conditional statements (`if-else`). Bottom Mechanism: Because threads in a Warp share the same instruction pipeline, different execution branches must be serialized. The hardware uses an Active Mask register to disable non-participating threads for each branch. Developers must align branch boundaries to 32-thread intervals to avoid serial stalls.

[M2.1] GPU Latency Hiding & Occupancy Optimization

Core Definition: Occupancy is the ratio of active Warps per SM to the maximum theoretical active Warps supported by the hardware. Bottom Mechanism: GPUs lack complex branch predictors or massive caches; instead, they rely on zero-overhead context switching. When a Warp stalls (e.g., waiting for global memory), the scheduler instantly switches to a ready Warp, hiding latency. High occupancy ensures a large pool of ready Warps.

[M2.2] GPU Memory Hierarchy & Latency

Core Definition: The memory hierarchy ranges from registers, local memory, shared memory, constant memory, texture memory, L1/L2 caches, to global memory. Bottom Mechanism: Registers feature 1-cycle access; on-chip Shared Memory takes tens of cycles; Global Memory takes 400 - 800 cycles. High-performance CUDA design centers on staging data from global memory into high-speed on-chip SRAM (Shared Memory) for reuse.

[M2.3] Global Memory Coalescing

Core Definition: When threads in a Warp access global memory, the hardware merges 32 memory requests into a single or minimum number of memory transactions. Bottom Mechanism: Memory controllers fetch data in aligned segments of 32, 64, or 128 bytes. If thread accesses are contiguous and aligned, one transaction serves the Warp. Uncoalesced accesses force up to 32 separate transactions, bottlenecking the memory bus.

[M2.4] Shared Memory & 32-Bank Organization

Core Definition: Shared Memory is a fast on-chip SRAM allocated per Block, structured physically into 32 independent memory banks. Bottom Mechanism: Successive 32-bit (or 64-bit) words are mapped cyclicly to Bank 0 through Bank 31. If the 32 threads of a Warp access 32 distinct banks in the same cycle, the access executes in parallel. This bank architecture yields multi-terabyte-per-second aggregate bandwidth.

[M2.5] Shared Memory Bank Conflict & Broadcast

Core Definition: A Bank Conflict occurs when multiple threads in a Warp access different addresses within the same memory bank in the same cycle. Bottom Mechanism: Accesses to the same bank must be serialized (an n-way conflict increases latency by n times). Exception: if all threads in a Warp access the same address in a bank, the hardware performs a Broadcast, serving all threads in 1 cycle without conflict. Use odd strides to avoid conflicts.

L1 Logical Framework

Grid Topology Mapping: The software logical layer abstracts computation tasks into Grid and Block, where millions of lightweight threads are mapped to 1D/2D physical data arrays in a 3D coordinate system via `threadIdx` and `blockIdx`. SIMT Hardware Scheduling: The hardware execution layer maps thread Blocks to Streaming Multiprocessors (SMs), where thread warps (32 threads) serve as the basic physical scheduling and instruction issue unit. High-Throughput Storage Hierarchy: A multi-tiered high-throughput memory hierarchy consisting of thread-private registers, Block-shared memory (SRAM), and Global Memory (GDDR/HBM) is constructed, optimized via coalesced global memory access and bank-conflict-free shared memory access. Async Pipeline Concurrency: Intra-block thread synchronization is enforced via `__syncthreads()`, while tasks and memory copies are overlapped asynchronously using CUDA Streams and CUDA Graphs to maximize hardware utilization.

L2 Data Flow Topology

GPU Host Memory → (PCIe / NVLink Copy HtoD) → GPU Global Memory → (Grid Launch) → SM Cache / Registers → (Shared Memory Intra-Block Reduction) → Warp Registers for Arithmetic (FMA / Tensor Core) → (Shared Memory / Global Memory Writeback) → (PCIe / NVLink Copy DtoH) → CPU Host Memory Result

Trade-off Matrix: Pipeline Overhead & Accuracy

Memory Bandwidth: Low to Moderate (100 - 300 GB/s) → High (1.5 - 2.5 TB/s) [Extremely High (Intra-register swap, zero bus latency)] On-chip Memory Overhead: 0 Bytes (No explicit SRAM requested) → 16KB - 96KB / Block [0 Bytes (Directly reuse general Register File)] Sync Overhead: None (Independent thread execution) → Moderate (Requires `__syncthreads()` barrier) [Extremely Low (Only warp-level sync)] Development Complexity: Extremely Low (Simple 1D/Multi-D pointer index) → Moderate to High (Requires manual tiling, collaborative load, sync) [High (Requires bitwise shifts, masks, and warp alignment)]

[M3.1] Register Spilling Overhead

Core Definition: Register Spilling occurs when a thread requests more local variables than the hardware register allocation limit per thread. Bottom Mechanism: The compiler forces excess variables into Local Memory (physically located in Global Memory). Although cached in L1, Local Memory access is hundreds of times slower than register access, leading to severe throughput degradation.

[M3.2] Constant & Texture Memory Caching

Core Definition: Constant and Texture memories are read-only memory spaces cached at the SM level. Bottom Mechanism: Constant memory excels when all threads of a Warp read the same address simultaneously (single-cycle broadcast). Texture memory features 2D spatial locality caching and dedicated hardware for filtering/clamping, ideal for graphics and spatial data.

[M3.3] L1 / L2 Cache Configuration

\\textbullet\\textbf{Core Definition:} The L1 cache and Shared Memory share a physical SRAM pool within each SM, allowing dynamic allocation sizing. \\textbullet\\textbf{Bottom Mechanism:} Developers can configure the SRAM partitioning (e.g., 96KB Shared / 32KB L1 or 64KB Shared / 64KB L1) via APIs. The L2 cache is a unified global cache that buffers non-coalesced accesses, mitigating memory bus congestion.

[M3.4] Unified Memory Page Migration

\\textbullet\\textbf{Core Definition:} Unified Memory (UM) provides a single, unified virtual address space (UVA) allowing the CPU and GPU to share pointers with automatic data movement. \\textbullet\\textbf{Bottom Mechanism:} When the GPU accesses an address that is not physically present in VRAM, it triggers a page fault. The driver intercepts the fault and asynchronously migrates the corresponding 4KB virtual pages over PCIe/NVLink.

[M3.5] Barriers & Memory Fences

\\textbullet\\textbf{Core Definition:} Synchronizations govern execution ordering and memory visibility across GPU threads. \\textbullet\\textbf{Bottom Mechanism:} `__syncthreads()` acts as an execution barrier, halting all threads in a Block until everyone reaches it. `__threadfence()` acts as a memory fence; it does not block execution but flushes outstanding writes to ensure global visibility before proceeding.

[M4.1] Cooperative Groups Programming Model

\\textbullet\\textbf{Core Definition:} Cooperative Groups provide a clean abstraction for defining and synchronizing custom groups of threads (sub-blocks or grid-wide). \\textbullet\\textbf{Bottom Mechanism:} It breaks the block-wide constraint of `__syncthreads()`. Developers can group threads by Warp tiling (e.g., groups of 8/16) or synchronize globally across the entire Grid using `this_grid()`, enabling safe multi-block iteration.

[M4.2] Warp Shuffle Register Data Exchange

\\textbullet\\textbf{Core Definition:} Warp Shuffle operations allow threads within a Warp to read registers from other threads in the same Warp directly. \\textbullet\\textbf{Bottom Mechanism:} Registers are read directly via crossbar routing inside the Register File, bypassing Shared Memory and L1 cache entirely. This eliminates the need for execution barriers and is highly optimized in primitives like `__shfl_sync` and `__shfl_down_sync`.

[M4.3] GPU Atomic Operations & L2 ALU Acceleration

\\textbullet\\textbf{Core Definition:} Atomic operations (e.g., `atomicAdd`) guarantee that read-modify-write actions on shared or global memory are executed atomically. \\textbullet\\textbf{Bottom Mechanism:} Modern GPUs integrate hardware atomic ALUs directly into the L2 cache controllers. Global atomics are executed at the L2 cache line level, avoiding costly round-trips to registers and minimizing lock contention in high-concurrency writes.

[M4.4] CUDA Streams & Concurrency Model

\\textbullet\\textbf{Core Definition:} A CUDA Stream represents a queue of commands (kernels, memcpys) that execute sequentially on the GPU. \\textbullet\\textbf{Bottom Mechanism:} The default stream (Stream 0) is synchronous and blocks operations on other streams. Custom non-default streams execute concurrently and out-of-order, allowing overlap of memory copy and kernel execution across different streams.

[M5.1] Host-Device Synchronization & Events

\\textbullet\\textbf{Core Definition:} GPU operations are asynchronous to the CPU; Events monitor stream execution progress and measure time intervals. \\textbullet\\textbf{Bottom Mechanism:} `cudaEventRecord` places an event marker in a stream. When the GPU reaches it, the event is marked complete. The host queries the event status via `cudaEventQuery` or blocks CPU execution using `cudaEventSynchronize` until the stream drains.

[M5.2] CUDA Graphs Task Graph Launch

\\textbullet\\textbf{Core Definition:} CUDA Graphs model workflow launches (kernels, memcpys) as a static Directed Acyclic Graph (DAG) instead of individual API calls. \\textbullet\\textbf{Bottom Mechanism:} Traditional launch sequences incur high CPU overhead (driver validation and launch latency of 3-5 microseconds per kernel). A captured Graph packages these nodes, sending a single execution token to the GPU, removing host launch overhead.

[M5.3] Page-locked (Pinned) Memory & DMA Transfer

\\textbullet\\textbf{Core Definition:} Pinned memory is host RAM allocated such that the OS cannot swap it out to virtual memory. \\textbullet\\textbf{Bottom Mechanism:} Allocated using `cudaMallocHost`, this memory is locked in physical RAM. The GPU's DMA (Direct Memory Access) engine can bypass the CPU and read host RAM directly, reaching the physical limits of the PCIe/NVLink bus width.

[M5.4] Multi-GPU Peer-to-Peer (P2P) NVLink Mesh

\\textbullet\\textbf{Core Definition:} Peer-to-Peer (P2P) enables GPUs in a system to directly read and write each other's VRAM without copying to host RAM. \\textbullet\\textbf{Bottom Mechanism:} Using high-bandwidth NVLink or NVSwitch interconnections, peer GPUs bypass the PCIe slot and CPU altogether, communicating directly at high speed. This forms the backbone of modern distributed large-model training clusters.

[M5.5] Tensor Cores MMA Hardware Acceleration

\\textbullet\\textbf{Core Definition:} Tensor Cores are specialized execution units designed to accelerate matrix multiply-accumulate operations in hardware. \\textbullet\\textbf{Bottom Mechanism:} In a single clock cycle, a Tensor Core performs a small matrix multiplication ($D = A \times B + C$, e.g., 16x16x16 shape) at the ASIC level. By hardwiring arithmetic arrays, they process massive Multiply-Accumulate operations far faster than standard SP cores.

[M6.1] CUDA Dynamic Parallelism

\\textbullet\\textbf{Core Definition:} Dynamic Parallelism enables a GPU Kernel to launch other child Kernels directly on the device without CPU intervention. \\textbullet\\textbf{Bottom Mechanism:} The GPU's internal device-side scheduler handles child kernel execution parameters directly. This allows highly efficient recursive algorithms, quad-tree refinement, and adaptive mesh refinement inside device threads.

[M6.2] GPU Performance Optimization Matrix

\\textbullet\\textbf{Core Definition:} Strategies to diagnose and solve Memory-bound and Compute-bound bottlenecks. \\textbullet\\textbf{Bottom Mechanism:} Use Roofline analysis to measure arithmetic intensity (FLOPs per byte transferred). Memory-bound kernels require coalescing and shared memory tiling. Compute-bound kernels require loop unrolling, fast-math intrinsics, and Tensor Core utilization.

[M6.3] GPGPU Parallel Patterns (Reduction/Scan/GEMM)

\\textbullet\\textbf{Core Definition:} Classical algorithmic paradigms for massive parallel computing. \\textbullet\\textbf{Bottom Mechanism:} In Parallel Reduction, warp-level unrolling and tree-reduction are used to avoid block-wide synchronization. In Histogram generation, Shared Memory local atomics are used to buffer writes, resolving heavy contention on Global Memory.

[M6.4] Nsight Compute & Systems Diagnostics

\\textbullet\\textbf{Core Definition:} NVIDIA's official profiling suite for system-level and kernel-level performance tuning. \\textbullet\\textbf{Bottom Mechanism:} Nsight Systems tracks global timelines, showing CPU-GPU stream overlaps and API delays. Nsight Compute gathers detailed Performance Monitor (PM) counters, reporting SM occupancy, Speed of Light (SOL), and pipeline stalls.

Zone T1: CUDA APIs & Primitive Keywords

: Device/Host Keywords & Primitive API Quick Reference

Zone T2: Errors & Hardware Stall Dictionary

: Runtime Errors & Hardware Stall Diagnosis Dictionary